Module 03: Feature Engineering I - Interaction Features and Continuous Binning

Cybernauts 2026 Datathon Campaign

May 2026

Abstract

Machine Learning models are powerful, but they aren't magic. Sometimes, the most important pattern in your data isn't in a single column, but in the **relationship** between two columns. Other times, the exact numerical value of a feature is just "noise," and the model would perform better if it saw broad categories instead. This note covers how to engineer interaction features and how to simplify continuous data through binning.

Contents

1 Interaction Features: The Force Multipliers

An interaction feature is a new column created by mathematically combining two or more existing ones.

Why Interactions Matter

While complex models like Gradient Boosted Trees can eventually "discover" these relationships on their own, giving them the relationship explicitly makes the learning process much faster and more robust. It turns a non-linear problem into a linear one.

1.1 Common Types of Interactions

- **Ratios (Division):** Used to normalize values. *Example:* If you have `Total_Calories` and `Serving_Size`, the model will benefit from:

$$\text{Calories\_Per\_Gram} = \frac{\text{Total\_Calories}}{\text{Serving\_Size}} \tag{1}$$

- **Products (Multiplication):** Used when the effect of one variable depends on the other. *Example:* `House_Area` $\times$ `Price_Per_Sqft`.
- **Differences (Subtraction):** Used to measure a "gap" or "change." *Example:* `Price_Listed` - `Price_Sold`.

```
# Creating a ratio feature in Pandas
df['Efficiency'] = df['Output'] / df['Input']

```

2 Continuous Binning (Quantization)

Binning is the process of turning a continuous number (like Age or Income) into categorical "buckets" or "bins."

The Generalization Benefit

Continuous data often contains small, random fluctuations that are just noise. For example, is there a real difference between someone who is 25.4 years old and 25.6 years old? Probably not. By binning them into a [19–35] bucket, you help the model ignore the noise and focus on the meaningful age group.

2.1 Implementation with `pd.cut`

Pandas provides the `pd.cut()` function to segment data into specific ranges.

```
# Binning Age into 4 categories
bins = [0, 18, 35, 60, 100]
labels = ['Minor', 'Young Adult', 'Middle Aged', 'Senior']
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)

```

2.2 Quantile-based Binning with `pd.qcut`

If you want each bin to have the **same number of people** (e.g., separating the dataset into 4 equal quartiles), use `qcut`.

```
# Divide Income into 4 equal groups (Quartiles)
df['Income_Tier'] = pd.qcut(df['Income'], q=4, labels=['Q1', 'Q2', 'Q3', '
Q4'])

```

3 Practice Problems

P1: You are building a model to predict house prices. You have `Lot_Frontage` and `Lot_Depth`. What interaction feature could you create that represents the total size of the lot?
Answer: $\texttt{Lot\_Area} = \texttt{Lot\_Frontage} \times \texttt{Lot\_Depth}$.

P2: You have a column `Exam_Score` (0–100). You notice that the difference between a 91 and a 93 doesn't matter, but the difference between an 89 and a 91 (the "A" threshold) matters a lot. Should you keep the scores as numbers or bin them?
Answer: Bin them into grade categories (e.g., F, D, C, B, A). This focuses the model on the meaningful thresholds rather than the numerical noise.

P3: What is the difference between `pd.cut` and `pd.qcut`?
Answer: ***`pd.cut`*** *creates bins based on specific* ***value ranges*** *(e.g., 0-10, 10-20), while* ***`pd.qcut`*** *creates bins based on* ***quantiles****, ensuring each bin has roughly the same number of data samples.*